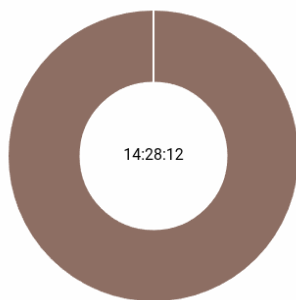


PRACTICA 2 SWAP: BALANCEADORES DE CARGA

ÁLVARO GARCÍA ZAFRA



Algunos de los problemas de que se me haya alargado tanto el tiempo ha sido el problema que me ha surgido en el avanzado 5 y que no entendí bien el 3 e iba a desarrollar 2 nuevos contenedores, teniendo un problema con el segundo de ellos que no era capaz de que funcionara

ASPECTOS BÁSICOS

Tareas Básicas - B1. Configuración del Entorno

El primer paso que he llevado a cabo en la práctica ha sido la creación de los directorios para los balanceadores, por un lado he creado uno para los archivos de configuración de Nginx y por otro lado he creado otro para los archivos de configuración de HAProxy.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass$ mkdir P2
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass$ cd P2
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P2$ mkdir P2-algarza04-nginx
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P2$ mkdir P2-algarza04-haproxy
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P2$ cp -r ../P1/P1-apache/web_algarza04/ P2-algarza04-nginx/
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P2$ cp -r ../P1/P1-apache/web_algarza04/ P2-algarza04-haproxy/
```

A continuación podemos ver el esquema final de ambos directorios.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P2$ tree P2-algarza04-nginx/
P2-algarza04-nginx/
├── docker-compose.yml
├── DockerfileApache_algarza04
├── DockerfileNginx_algarza04
├── nginx.conf
├── web_algarza04
│   └── index.php
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicass/P2$ tree P2-algarza04-haproxy/
P2-algarza04-haproxy/
├── docker-compose.yml
├── DockerfileApache_algarza04
├── DockerfileHAProxy_algarza04
├── haproxy.cfg
├── web_algarza04
│   └── index.php
```


He abierto una terminal de este contenedor y no me ha dado problema, además también me abre su página, por lo que parece que el Dockerfile es correcto.

Cuando cree el nginx.conf y me dispuse a comprobar su funcionamiento pude ver que había algunos fallos en su creación, estos fallos se debían a ciertos cambios que debía realizarle al fichero, entre ellos añadir una serie de cabeceras, añadir el apartado events y eliminar el algoritmo que le estaba pasando(round-robin) ya que es el que se usa actualmente por defecto.

nginx.conf

```
user nginx;
worker_processes auto;
error_log /var/log/nginx/error.log;
pid /var/run/nginx.pid;

events {
    worker_connections 1024;
}

http {
    upstream backend_algarza04 {
        server 192.168.10.2;
        server 192.168.10.3;
        server 192.168.10.4;
        server 192.168.10.5;
        server 192.168.10.6;
        server 192.168.10.7;
        server 192.168.10.8;
        server 192.168.10.9;
    }
    server {
        listen 80;
        server_name nginx_algarza04;

        access_log /var/log/nginx/nginx_algarza04.access.log;
        error_log /var/log/nginx/nginx_algarza04.error.log;
        location / {
            proxy_pass http://backend_algarza04;
            proxy_set_header Cookie $http_cookie;
            proxy_hide_header Set-Cookie;
        }
        location /estadisticas_algarza04 {
            stub_status on;
        }
    }
}
```

- En primer lugar nos encontramos con una serie de cabeceras, la primera hace que el servidor se ejecute como el usuario nginx, la segunda que el número de procesos de trabajo se ajuste en función de los núcleos del sistema y las dos últimas indican donde guardar los errores y el pid.
- El apartado eventos indica que cada proceso puede manejar 1024 conexiones simultáneas.
- El apartado http se divide en dos bloques a su vez:
 - El upstream define el grupo de servidores entre los que se balanceará la carga, estos son todos los servidores web que se crearán con el docker-compose, además como no indicamos que balanceo queremos se aplicará por defecto round-robin.
 - Por otro lado tenemos el bloque server, el cual se encargará del comportamiento del servidor, este escucha el puerto 80 y crea un servidor

virtual que almacena los accesos y errores. También hay un bloque location que redirige las peticiones a uno de los servidores del upstream, este a su vez tiene dentro el archivo que guardará las estadísticas del servidor.

Tareas Básicas - B3. Implementación de Nginx con Docker Compose

En este apartado me serví de la ayuda de la ia para saber como darle un nombre por defecto a las redes que se iban a crear (red_web y res_servicios) para aprovecharlas para los siguientes servicios a crear(HAProxy).

Los dockerfile de apache y su creación en el docker-compose las he copiado de la práctica anterior, por lo que no los he explicado en ningún lugar de esta.

```
networks:
  red_web:
    name: red_web
    ipam:
      config:
        - subnet: 192.168.10.0/24
  red_servicios:
    name: red_servicios
    ipam:
      config:
        - subnet: 192.168.20.0/24
```

En este docker-compose he añadido un parámetro para que las redes se creen con un nombre por defecto y así poderlas aprovecharlas para los siguientes servidores.

```
balanceador-nginx:
  build:
    context: .
    dockerfile: DockerfileNginx_algarza04
  image: algarza04-nginx-image:p2
  container_name: nginx
  ports:
    - 8090:80
  volumes:
    - ./nginx.conf:/etc/nginx/nginx.conf
  networks:
    red_web:
      ipv4_address: 192.168.10.50
  depends_on:
    - web1
    - web2
    - web3
    - web4
    - web5
    - web6
    - web7
    - web8
```

- Se crea el balanceador en el directorio actual y a partir del Dockerfile creado para Nginx.
- Crea la imagen y el contenedor con los nombres que se le han pasado.
- Mapea el puerto 80 del contenedor al 8090 del host.
- En el apartado volumes se monta el .conf que hemos creado dentro del contenedor, sobrescribiendo el original.
- En networks se le asigna una red fija.
- Por último le indica que el balanceador se iniciará una vez estén todos los servidores nombrados iniciados.

Tareas Básicas - B4. Verificación y Pruebas de Nginx

Una vez hemos terminado de realizar todos los archivos de configuración necesarios para levantar los servidores y el balanceador vamos a pasar a levantar estos, para ello usamos `$docker compose up -d`(como ya vimos en la práctica anterior el flag `-d` sirve para la ejecución de este en segundo plano).

Una vez realizado esto los servicios deberían de haber sido creados y levantados correctamente.

```
[+] Running 20/20
✓ balanceador-nginx          Built
✓ web1                       Built
✓ web2                       Built
✓ web3                       Built
✓ web4                       Built
✓ web5                       Built
✓ web6                       Built
✓ web7                       Built
✓ web8                       Built
✓ Network p2-algarza04-nginx_red_web    Created
✓ Network p2-algarza04-nginx_red_servicios Created
✓ Container web3              Started
✓ Container web5              Started
✓ Container web2              Started
✓ Container web6              Started
✓ Container web8              Started
✓ Container web4              Started
✓ Container web1              Started
✓ Container web7              Started
✓ Container nginx             Started
```

Cuando yo lo hice de primeras me dio problemas por algo relacionado con las imágenes que tenía creadas en ese momento, por lo que las borré todas.

Las siguientes imágenes nos van a servir para verificar que se han creado correctamente los servicios y que el balanceador los ha creado en el orden correcto(el orden de las imágenes es por orden de salida, es decir, el mismo orden en el que el balanceador fue repartiendo la carga)

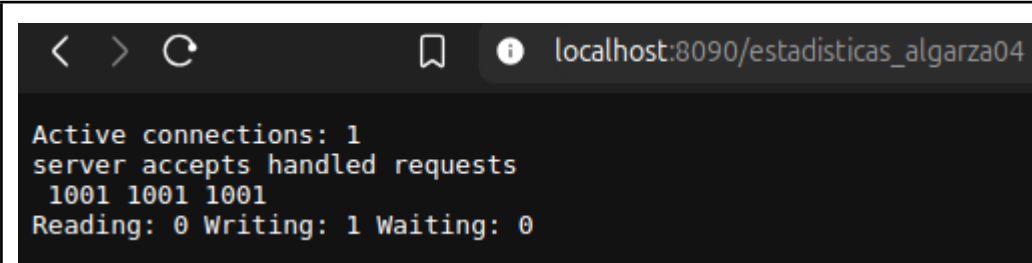
 localhost:8090 SWAP - "Álvaro García Zafra" Dirección IP del servidor: 192.168.10.2	 localhost:8090 SWAP - "Álvaro García Zafra" Dirección IP del servidor: 192.168.10.3
--	---

Además también realice un script que mostrará el número de solicitudes que había atendido cada servidor.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-nginx$ ./contadorSolicitudes.sh
web1: 1
web2: 1
web3: 1
web4: 1
web5: 1
web6: 1
web7: 1
web8: 1
```

Para comprobar si el balanceador está mandando todas las solicitudes y se están reflejando de manera correcta en las estadísticas vamos a utilizar el comando `$ab -n 1000 -c 10 http://localhost:8090/`, este se encargará de enviar 1000 peticiones, concurrentemente de 10 en 10 al balanceador.



Como podemos ver en las estadísticas se muestra el resultado esperado, ya que muestra las 1000 solicitudes realizadas + 1 por la petición de ver las estadísticas, además, si analizamos la salida podemos ver que hay una conexión activa (las estadísticas), que nginx ha aceptado 1001 peticiones, que ha manejado 1001 y que HTTP ha procesado 1001 solicitudes.

Tareas Básicas - B5. Configuración de HAProxy como Balanceador de Carga

Esta tarea consiste en la creación del Dockerfile y del .conf de HAProxy, por lo que a continuación voy a pasar a analizar que le he metido a cada uno de ellos. En este apartado he utilizado la ia para averiguar cual es la localización del archivo de configuración por defecto (/etc/haproxy/haproxy.cfg) aunque finalmente vi que en el guión había una y puse esa, también me sirvió de su ayuda para averiguar cuál es el CMD necesario.

```
FROM haproxy:latest

EXPOSE 80

CMD ["haproxy", "-f", "/usr/local/etc/haproxy/haproxy.cfg", "-db"]
```

- El from indica que se va a construir el contenedor en base a la última imagen oficial de HAProxy.
- Expone el puerto 80 dentro del contenedor.
- El CMD funciona de la siguiente manera, haproxy se encarga de ejecutar su binario, posteriormente el -f /usr... indica cuál debe ser el archivo de configuración que debe usar, en este caso será el que hemos creado nosotros ya que se ha copiado allí, por último añadí -db que hace que el contenedor se ejecute en primer plano y no se pare nada más ejecutarlo, lo cuál era un problema que tenía nada más crearlo.

Una vez creado pasé a comprobar si era correcto:

```
alvarogz@agz-ubu24 ~/Escritorio/SMAP/practicass/P2/P2-algarza04-haproxy$ docker build -t haproxy-image -f DockerfileHAProxy_algarza04 .
[+] Building 4.2s (7/7) FINISHED
=> [internal] load build definition from DockerfileHAProxy_algarza04
=> [internal] load metadata for docker.io/library/haproxy:latest
=> [internal] load .dockerignore
=> [internal] load build context
=> [1/2] FROM docker.io/library/haproxy:latest@sha256:16415274ffb34bd08043c61ecdf4da6231b0843b020c06a0621c17463043
=> resolve docker.io/library/haproxy:latest@sha256:16415274ffb34bd08043c61ecdf4da6231b0843b020c06a0621c17463043
=> sha256:42c372f5ae0415559dd84f7737ecfb01e4812db7d9559fb22a06c4611b6cd7cd 2.10kB / 2.10kB
=> sha256:7d0cf6a78796719086e9577a9c086906e2247d83a2dfbf258d88d9941e0406f3 1.16kB / 1.16kB
=> sha256:adc835e4fc840b4c3e3db2c43cbd119f8f3bd60879c5f73036fd0de0d12768ab 10.05MB / 10.05MB
=> sha256:16415274ffb34bd08043c61ecdf4da6231b0843b020c06a0621c174630437a7e 10.18kB / 10.18kB
=> sha256:d1974ef943cbd4be3918b4d3ce0f4dbfd94e2ffe3c574943f5fe950ca5dd84f 4.74kB / 4.74kB
=> sha256:64ad64f8fe64e8d6fcb790b7f50f63c9b0707d74669a24c91e45b69326acde5c 3.50MB / 3.50MB
=> sha256:43a8e5df452aae064702d86375f53ba3ff527a2e3833e797ca166ee0949c056 452B / 452B
=> extracting sha256:64ad64f8fe64e8d6fcb790b7f50f63c9b0707d74669a24c91e45b69326acde5c
=> sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1 32B / 32B
=> extracting sha256:7d0cf6a78796719086e9577a9c086906e2247d83a2dfbf258d88d9941e0406f3
=> extracting sha256:adc835e4fc840b4c3e3db2c43cbd119f8f3bd60879c5f73036fd0de0d12768ab
=> extracting sha256:43a8e5df452aae064702d86375f53ba3ff527a2e3833e797ca166ee0949c056
=> extracting sha256:4f4fb700ef54461cfa02571ae0db9a0dc1e0c0b5577484a6d75e68dc38e8acc1
=> [2/2] COPY haproxy.cfg /etc/haproxy/haproxy.cfg
=> exporting to image
=> exporting layers
=> writing image sha256:8348b6383f970208b4560e20342fbed1726c5d5a928369ff7b8556d3c28563
=> naming to docker.io/library/haproxy-image
```

REPOSITORY	TAG	IMAGE ID	CREATED	SIZE
haproxy-image	latest	8348b6383f97	36 seconds ago	105MB

Como podemos ver se ha creado la imagen correctamente, por lo que el siguiente paso es levantar un contenedor con dicha imagen.


```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-haproxy$ docker run -d -p 8091:80 --name haproxy-prueba haproxy-image:latest
0c1c2468067ca167222a2c504e1accaa77a8132ac47a2af7ac3f06ac0fc8549d
```

Al hacer esto el contenedor no se quedaba levantado, por lo que investigué y le añadí un nuevo parámetro a su CMD(db) y de esta manera conseguí que se quedará levantado.

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-haproxy$ docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
8918342152a4   haproxy-image:latest   "docker-entrypoint.s..."   5 seconds ago   Up 4 seconds   0.0.0.0:8091->80/tcp, [::]:8091->80/tcp   haproxy-prueba

alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-haproxy$ docker exec -it haproxy-prueba /bin/sh
$
```

He abierto una terminal de este contenedor y no me ha dado problema, por lo que parece que el Dockerfile es correcto.

```
frontend frontend_algarza04
    bind *:80
    default_backend backend_algarza04

backend backend_algarza04
    server web1 192.168.10.2:80 maxconn 32
    server web2 192.168.10.3:80 maxconn 32
    server web3 192.168.10.4:80 maxconn 32
    server web4 192.168.10.5:80 maxconn 32
    server web5 192.168.10.6:80 maxconn 32
    server web6 192.168.10.7:80 maxconn 32
    server web7 192.168.10.8:80 maxconn 32
    server web8 192.168.10.9:80 maxconn 32

global
    stats socket /var/lib/haproxy/stats

listen stats
    bind *:9000
    mode http
    stats enable
    stats uri /estadisticas_algarza04
    stats realm HAProxy\ Statistics
    stats auth algarza04:SWAP1234
```

- En el frontend se define el punto de entrada, indica que va a escuchar todas las interfaces en el puerto 80 y que redirige el tráfico hacia el backend_algarza04.
- En este backend_algarza04 se definen todos los servidores hacia los que el balanceador va a enviar la carga, estos pueden manejar hasta 32 conexiones simultáneas, como no hemos puesto el método de balanceo usará por defecto round-robin.
- Los dos últimos bloques son los que se encargarán de las estadísticas, expondrá la interfaz al puerto 9000, define la ruta, el nombre del panel y el usuario y contraseña.

Tareas Básicas - B6. Implementación de HAProxy con Docker Compose

En este apartado hemos creado el docker-compose, en este he establecido las redes como externas aprovechando así las redes creadas anteriormente con nginx.

Al igual que en Nginx la configuración de apache es idéntica a la práctica anterior.

```
networks:
  red_servicios:
    external: true
  red_web:
    external: true
```

```
balanceador-haproxy:
  build:
    context: .
    dockerfile: DockerfileHAProxy_algarza04
  image: algarza04-haproxy-image:p2
  container_name: haproxy
  ports:
    - 8091:80
    - 9000:9000
  volumes:
    - ./haproxy.cfg:/usr/local/etc/haproxy/haproxy.cfg
  networks:
    red_web:
      ipv4_address: 192.168.10.50
  depends_on:
    - web1
    - web2
    - web3
    - web4
    - web5
    - web6
    - web7
    - web8
```

- Este balanceador sigue la misma estructura que el anterior, se crea con el dockerfile creado de haproxy, se le da el nombre a la imagen y el contenedor, le asignamos la red y hacemos que se cree una vez estén creadas las 8 redes web.
- Al contrario que el anterior, aparte del puerto para acceder al balanceador tiene un puerto para acceder a las estadísticas.
- En el apartado volumes se monta el .cnf que hemos creado dentro del contenedor, sobrescribiendo el original.

Tareas Básicas - B7. Verificación y Pruebas de HAProxy

Para realizar este apartado he ejecutado el comando `$docker compose up -d`, como podemos ver en la siguiente imagen se han creado y levantado los contenedores de manera correcta, lo que nos indica a priori que los archivos de configuración son correctos.

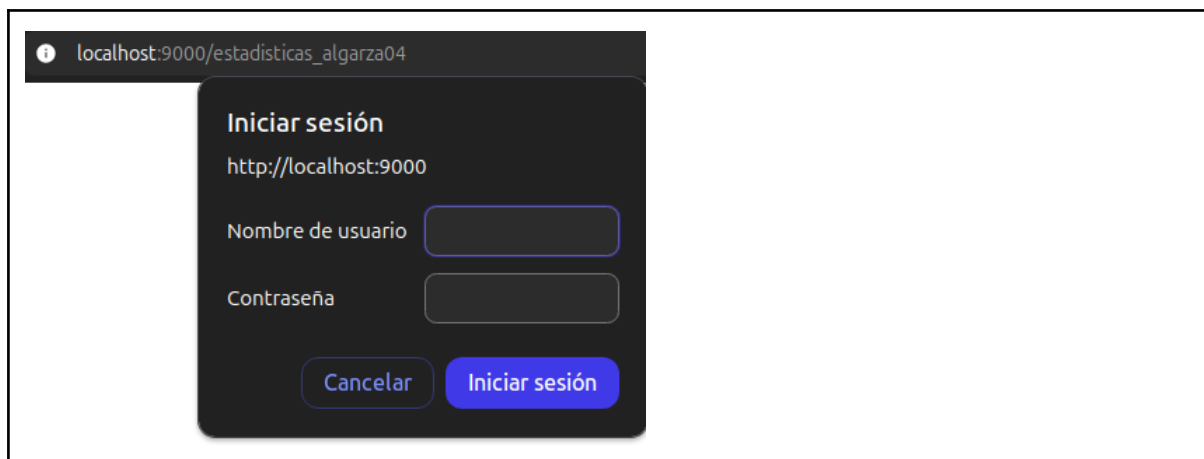
```
✓ balanceador-haproxy Built
✓ web1 Built
✓ web2 Built
✓ web3 Built
✓ web4 Built
✓ web5 Built
✓ web6 Built
✓ web7 Built
✓ web8 Built
✓ Container web4 Started
✓ Container web7 Started
✓ Container web6 Started
✓ Container web1 Started
✓ Container web8 Started
✓ Container web3 Started
✓ Container web5 Started
✓ Container web2 Started
✓ Container haproxy Started
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-haproxy$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        PORTS
6a99499b1664   algarza04-haproxy-image:p2         "docker-entrypoint.s..." 6 seconds ago   Up 5 seconds   0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp,
0.0.0.0:8091->80/tcp, [::]:8091->80/tcp
fb1818312dd8   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8086->80/tcp, [::]:8086->80/tcp
84b88edd39a9   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8084->80/tcp, [::]:8084->80/tcp
01d2637745c9   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8085->80/tcp, [::]:8085->80/tcp
d688b5124f31   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8087->80/tcp, [::]:8087->80/tcp
1d35b9f86912   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
664f46efff552   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
d01df075e4ec   algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
d49949dafa5    algarza04-apache-image:p2         "/bin/sh -c 'apache2..." 6 seconds ago   Up 6 seconds   0.0.0.0:8088->80/tcp, [::]:8088->80/tcp
```

Como podemos apreciar se han levantado correctamente los servidores y el balanceador.

En las siguientes capturas vamos a comprobar que desde el balanceador se reparte la carga a todos los servidores web y que es en orden, cosa que tiene que ser así debido a que es round-robin.

Para comprobar si el balanceador está mandando todas las solicitudes y se están reflejando de manera correcta en las estadísticas vamos a utilizar el comando `$ab -n 1000 -c 10 http://localhost:8091/`.



ASPECTOS AVANZADOS

Tareas avanzadas - A1. Configuraciones Avanzadas de Nginx

Existen diversas estrategias de balanceo, como he mencionado anteriormente yo he desarrollado balanceo con round-robin y a continuación voy a pasar a probar 2 nuevas estrategias, estas son balanceo ponderado y por menor tiempo de respuesta.

El balanceo ponderado consiste en aplicarle un peso a cada servidor, esto nos será muy útil cuando un servidor posea mayor carga de procesamiento que los demás.

El balanceo por menor tiempo de respuesta consiste en que se distribuyen las solicitudes al servidor con menos conexiones activas y con el tiempo de respuesta más rápido.

He creado un directorio en el cuál tendré los distintos tipos de balanceo creados y en función del que necesite se pondrá uno u otro en el docker-compose.

<pre>conf/ ├── menor tiempo │ └── nginx.conf ├── ponderado │ └── nginx.conf └── roundrobin └── nginx.conf</pre>	<pre>volumes: - ./conf/roundrobin/nginx.conf:/etc/nginx/nginx.conf</pre>
---	--

Para la realización de este ejercicio le he preguntado a la ia que atributos habría que añadirle al servidor, en el caso del ponderado es añadir un weight a cada uno, mientras que en el de menor tiempo es añadir la linea least_conn.

Para comprobar si se ha aplicado de manera correcta el balanceo por pesos voy a usar el script mencionado anteriormente que se encargaba de decirnos las conexiones realizadas por cada servidor.

En primer lugar he utilizado `$ab -n 1000 -c 10 http://localhost:8090/` para mandarle 1000 peticiones al balanceador y ver como las reparte, lo he hecho para ambos casos.

PONDERADO

```
upstream backend_algarza04 {
    server 192.168.10.2 weight=1;
    server 192.168.10.3 weight=1;
    server 192.168.10.4 weight=4;
    server 192.168.10.5 weight=2;
    server 192.168.10.6 weight=1;
    server 192.168.10.7 weight=8;
    server 192.168.10.8 weight=1;
    server 192.168.10.9 weight=1;
```

```
localhost:8090/estadisticas_algarza04

Active connections: 1
server accepts handled requests
1002 1002 1003
Reading: 0 Writing: 1 Waiting: 0

alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-nginx$ ./contadorSolicitudes.sh
web1: 54
web2: 53
web3: 211
web4: 105
web5: 53
web6: 421
web7: 52
web8: 51
```

Como podemos ver el número de solicitudes se ha realizado correctamente y a cada servidor se le ha enviado un número de solicitudes correspondido con los pesos de cada uno de ellos.

MENOR TIEMPO DE RESPUESTA

```
upstream backend_algarza04 {
    least_conn;
    server 192.168.10.2;
    server 192.168.10.3;
    server 192.168.10.4;
    server 192.168.10.5;
    server 192.168.10.6;
    server 192.168.10.7;
    server 192.168.10.8;
    server 192.168.10.9;
}
```

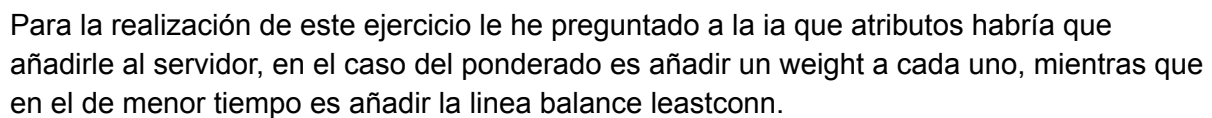
```
localhost:8090/estadisticas_algarza04

Active connections: 1
server accepts handled requests
1002 1002 1002
Reading: 0 Writing: 1 Waiting: 0
```

```
alvarogz@agz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-nginx$ ./contadorSolicitudes.sh
web1: 126
web2: 124
web3: 127
web4: 129
web5: 125
web6: 122
web7: 122
web8: 125
```

Como podemos ver en este caso el número de solicitudes también es correcto y se han repartido las cargas según su tiempo de respuesta.

Al igual que en el ejercicio anterior he creado un directorio con los distintos métodos de balanceo.



HAProxy version 3.1.7-c3f4089, released 2025/04/17

Statistics Report for pid 8

> General process information

```
pid = 8 (process #1, nproc = 1, rthread = 8)
uptime = 0.0400116s, warnings = 3
system limits: memmax = unlimited, limit-nr = 1048576
memsize = 1048576, maxconn = 524288, reached = 0, maxfiles = 0
current conn = 1, current pipes = 0/0, conn rate = 0/sec, bit rate = 0.00 kbps
Running tasks: 0/23 (0 nice); idle = 100 %
```

active UP
 active UP going down
 active DOWN, going up
 active or backup DOWN
 active or backup DOWN for maintenance (MAINT)
 active or backup SOFT STOPPED for maintenance

backup UP
 backup UP going down
 backup DOWN, going up
 not checked
 not checked
 Note: "NOLB"/"DRAIN" = UP with load-balancing disabled.

Display option:

- Scope : [redacted]
- [Hide DOWN servers](#)
- [Refresh lists](#)
- [CSV export](#)
- [JSON export \(schema\)](#)

External resources:

- [Primary site](#)
- [Updates \(V3.1\)](#)
- [Online manual](#)

frontend algarza04																																												
Queue					Session rate					Sessions					Bytes					Denied					Errors					Warnings					Status					Server				
Curr	Max	Limit	Cur	Max	Limit	Curr	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwtime	Thrtle															
					0	1	004	-	0	17	524	264	1	004						82	000	274	000	0	0	0	0	0	0	0	OPEN													

backend algarza04																																												
Queue					Session rate					Sessions					Bytes					Denied					Errors					Warnings					Status					Server				
Curr	Max	Limit	Cur	Max	Limit	Curr	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwtime	Thrtle															
web1	0	0	-	0	53	0	0	1	32	53	53	1m11s	4	346	14	522	0	0	0	0	no check	1/1	Y	-	-	-	-	-	-	-														
web2	0	0	-	0	53	0	0	2	32	53	53	1m11s	4	346	14	522	0	0	0	0	no check	1/1	Y	-	-	-	-	-	-	-														
web3	0	0	-	0	212	0	0	2	212	212	1m11s	17	302	57	814	0	0	0	0	no check	4/4	Y	-	-	-	-	-	-	-	-														
web4	0	0	-	0	106	0	0	2	32	106	106	1m11s	8	610	26	770	0	0	0	0	no check	2/2	Y	-	-	-	-	-	-	-														
web5	0	0	-	0	53	0	0	1	32	53	53	1m11s	4	264	14	248	0	0	0	0	no check	1/1	Y	-	-	-	-	-	-	-														
web6	0	0	-	0	423	0	0	7	32	423	423	1m11s	34	604	115	628	0	0	0	0	no check	8/8	Y	-	-	-	-	-	-	-														
web7	0	0	-	0	52	0	0	1	32	52	52	1m11s	4	264	14	248	0	0	0	0	no check	1/1	Y	-	-	-	-	-	-	-														
web8	0	0	-	0	52	0	0	2	32	52	52	1m11s	4	264	14	248	0	0	0	0	no check	1/1	Y	-	-	-	-	-	-	-														
Backend	0	0	-	0	1004	0	0	14	52	427	1004	1004	1m11s	82	000	274	000	0	0	0	0	1m17s UP	19/19	8	0	-	0	0s	-															

stats																																												
Queue					Session rate					Sessions					Bytes					Denied					Errors					Warnings					Status					Server				
Curr	Max	Limit	Cur	Max	Limit	Curr	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwtime	Thrtle															
					0	1	-	1	2	524	264	1	1849	60	391	0	0	0	0	0	0	0	OPEN																					
					0	0	-	0	0	52	427	0	0	1849	60	391	0	0	0																									

Como podemos ver el número de solicitudes se ha repartido en función del peso de cada uno, por lo que parece que está bien implementado.

MENOR TIEMPO DE RESPUESTA

```
backend backend_algarza04
balance leastconn
server web1 192.168.10.2:80 maxconn 32
server web2 192.168.10.3:80 maxconn 32
server web3 192.168.10.4:80 maxconn 32
server web4 192.168.10.5:80 maxconn 32
server web5 192.168.10.6:80 maxconn 32
server web6 192.168.10.7:80 maxconn 32
server web7 192.168.10.8:80 maxconn 32
server web8 192.168.10.9:80 maxconn 32
```

localhost:9000/estadisticas_algarza04

HAProxy version 3.1.7-c3f4089, released 2025/04/17

Statistics Report for pid 8

> General process information

pid = 8 (process #1, nbproc = 1, nbthread = 8)
uptime = 0d 0h00m07s; warnings = 3
system limits: memmax = unlimited; ulimit-n = 1048576
maxsock = 1048576; maxconn = 524264; reached = 0; maxpipes = 0
current conns = 1; current pipes = 0/0; conn rate = 1/sec; bit rate = 0.000 kbps
Running tasks: 0/29 (0 niced); idle = 99 %

active UP
active UP, going down
active DOWN, going up
active or backup DOWN
active or backup DOWN for maintenance (MAINT)
active or backup SOFT STOPPED for maintenance

backup UP
backup UP, going down
backup DOWN, going up
not checked

Display option:
• Scope:
• Hide DOWN servers
• Refresh now
• CSV export
• JSON export (schema)

External resources:
• Primary site
• Updates (v3.1)
• Online manual

frontend algarza04

Queue				Session rate				Sessions				Bytes				Denied		Errors		Warnings		Server									
Cur	Max	Limit		Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntime	Thrtle	
0	1	000	-	0	16		524 264	1 000					82 000	274 000	0	0	0					OPEN									

backend algarza04

Queue				Session rate				Sessions				Bytes				Denied		Errors		Warnings		Server									
Cur	Max	Limit		Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntime	Thrtle	
web1	0	0	-	0	124		0	2	32	124	124	3s	10 168	33 976		0	0	0	0	0	0	no check		1/1	Y	-					
web2	0	0	-	0	125		0	2	32	125	125	3s	10 250	34 250		0	0	0	0	0	0	no check		1/1	Y	-					
web3	0	0	-	0	128		0	3	32	128	128	3s	10 496	35 072		0	0	0	0	0	0	no check		1/1	Y	-					
web4	0	0	-	0	126		0	2	32	126	126	3s	10 332	34 524		0	0	0	0	0	0	no check		1/1	Y	-					
web5	0	0	-	0	124		0	2	32	124	124	3s	10 168	33 976		0	0	0	0	0	0	no check		1/1	Y	-					
web6	0	0	-	0	123		0	2	32	123	123	3s	10 086	33 702		0	0	0	0	0	0	no check		1/1	Y	-					
web7	0	0	-	0	125		0	2	32	125	125	3s	10 250	34 250		0	0	0	0	0	0	no check		1/1	Y	-					
web8	0	0	-	0	125		0	2	32	125	125	3s	10 250	34 250		0	0	0	0	0	0	no check		1/1	Y	-					
Backend	0	0		0	1 000		0	14	52 427	1 000	1 000	3s	82 000	274 000	0	0		0	0	0	0	8s UP		8/8	8	0		0	0s		

stats

Queue				Session rate				Sessions				Bytes				Denied		Errors		Warnings		Server									
Cur	Max	Limit		Cur	Max	Limit	Cur	Max	Limit	Total	LbTot	Last	In	Out	Req	Resp	Req	Conn	Resp	Retr	Redis	Status	LastChk	Wght	Act	Bck	Chk	Dwn	Dwntime	Thrtle	
1	1		-	1	1		1	1		524 264	1		0	0	0	0	0					OPEN									
0	0			0	0		0	0		52 427	0	0s	0	0	0	0	0		0	0	0	8s UP		0/0	0	0		0			

En este caso el número de solicitudes también es correcto y se han repartido las cargas según su tiempo de respuesta.

Tareas avanzadas - A3. Experimentación con Diferentes Balanceadores de Carga

En primer lugar he optado por desplegar un sistema con un balanceador Traefik, para esto he creado una serie de ficheros.

En primer lugar he creado un Dockerfile para crear el contenedor.

```
FROM traefik:latest
EXPOSE 80
EXPOSE 8080
CMD ["traefik"]
```

- Crea un contenedor traefik de la ultima versión y le expone sus puertos 80 y 8080, por último el cmd se encarga de buscar su configuración en el directorio por defecto.

```
entryPoints:
  web:
    address: ":80"

providers:
  docker:
    exposedByDefault: false

api:
  dashboard: true
  insecure: true
```

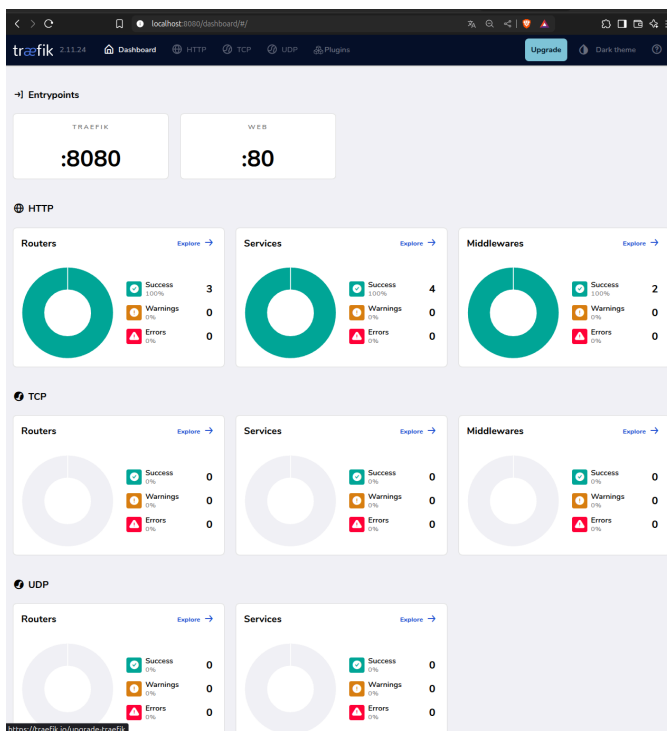
- entryPoints: permitirá el tráfico en el puerto 80.
- providers: por defecto no expondrá ningún contenedor, a menos que este indique lo contrario.
- api: habilita el dashboard y lo hace accesible sin autenticación.

```
balanceador-traefik:
  build:
    context: .
    dockerfile: DockerfileTraefik_algarza04
  image: algarza04-traefik-image:p2
  container_name: traefik
  ports:
    - 8090:80
    - 8080:8080
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - ./traefik.yml:/etc/traefik/traefik.yml
  networks:
    red_web:
      ipv4_address: 192.168.10.50
    red_servicios:
      ipv4_address: 192.168.20.50
  depends_on:
    - web1
    - web2
    - web3
    - web4
    - web5
    - web6
    - web7
    - web8
```

- Lo único que cambia respecto a anteriores docker compose es que añade el puerto 8080:8080 para el dashboard, influye varios volúmenes, el de configuración (.yml) y el docker.sock que permite que detecte contenedores en tiempo real.
- Otra cosa que lo diferencia de los anteriores es que necesita tener ambas redes conectadas para funcionar, ya que si no está en las dos le resulta inaccesible.

```
labels:
- "traefik.enable=true"
- "traefik.http.routers.web.rule=Host(`localhost`)"
- "traefik.http.services.web.loadbalancer.server.port=80"
```

- Otra diferencia es que cada web que quiera ser gestionada por traefik tiene que tener estos labels.
 - el primero permite que traefik gestione dicho contenedor.
 - el segundo hace que redirija la dirección al contenedor cuando se acceda a la web
 - el último hace que cuando le llegue tráfico al contenedor se conecte al puerto 80



En la anterior captura podemos ver como traefik está funcionando correctamente.

Voy a pasar a realizar una pequeña comparación entre Traefik y Nginx:

Traefik	Nginx
• Puedes configurarlo de manera dinámica. • Permite detectar un contenedor automáticamente. • Posee HTTPS de manera nativa. • Posee un dashboard • Muy bueno en entornos dinámicos	• Requiere que se recargue su archivo de configuración. • Hay que configurar los contenedores manualmente. • Hay que implementar HTTPS manualmente • No posee dashboard • Muy bueno en sitios con alto tráfico

Tareas avanzadas - A4. Investigación y Pruebas de Tolerancia a Fallos

En este apartado tenemos que probar la tolerancia a los fallos de nuestros servidores, para esto voy a apagar uno de los servidores web a los cuales el balanceador le mandaba carga para ver como es capaz de solucionarlo, voy a realizarlo tanto en Nginx como en HAProxy para ver como reasigna la carga a los demás.

Nginx

```
upstream backend_algarza04 {  
    server 192.168.10.2 weight=1 max_fails=3 fail_timeout=30s;  
    server 192.168.10.3 weight=1 max_fails=3 fail_timeout=30s;  
    server 192.168.10.4 weight=4 max_fails=3 fail_timeout=30s;  
    server 192.168.10.5 weight=2 max_fails=3 fail_timeout=30s;  
    server 192.168.10.6 weight=1 max_fails=3 fail_timeout=30s;  
    server 192.168.10.7 weight=8 max_fails=3 fail_timeout=30s;  
    server 192.168.10.8 weight=1 max_fails=3 fail_timeout=30s;  
    server 192.168.10.9 weight=1 max_fails=3 fail_timeout=30s;  
}
```

Le he añadido:

- `max_fails=3`: nos indica que el servidor puede fallar 3 veces consecutivas antes de que cuente como no disponible y lo descarte temporalmente.
- `fail_timeout=30s`: es el tiempo que estará el balanceador sin mandar nuevas peticiones al servidor.

Para comprobar si funciona correctamente he desconectado el servidor web6 que era al que más peticiones se le iba a hacer y me he puesto a realizar peticiones.

```
root@26bdb2e77a89:/# tail -f /var/log/nginx/nginx_algarza04.error.log  
2025/04/23 16:14:12 [error] 8#8: *1 connect() failed (113: No route to host) while connecting to upstream, client: 192.168.10.1,  
server: nginx_algarza04, request: "GET / HTTP/1.1", upstream: "http://192.168.10.7:80/", host: "localhost:8090"  
2025/04/23 16:14:20 [error] 8#8: *1 connect() failed (113: No route to host) while connecting to upstream, client: 192.168.10.1,  
server: nginx_algarza04, request: "GET / HTTP/1.1", upstream: "http://192.168.10.7:80/", host: "localhost:8090"  
2025/04/23 16:14:25 [error] 8#8: *1 connect() failed (113: No route to host) while connecting to upstream, client: 192.168.10.1,  
server: nginx_algarza04, request: "GET / HTTP/1.1", upstream: "http://192.168.10.7:80/", host: "localhost:8090"  
2025/04/23 16:14:59 [error] 9#9: *166 connect() failed (113: No route to host) while connecting to upstream, client: 192.168.10.  
1, server: nginx_algarza04, request: "GET / HTTP/1.1", upstream: "http://192.168.10.7:80/", host: "localhost:8090"
```

Posteriormente he mirado el archivo donde se han guardado todos los errores y en este podemos ver que los 3 primeros errores tienen poco tiempo entre uno y otro, este tiempo es el tiempo que ha tardado en volver a hacer una petición a este servidor, pero entre el tercer y el cuarto error vemos que hay más de 30 segundos de diferencia, esto se debe a lo que he comentado anteriormente, que una vez se han rechazado 3 peticiones se bloquea esa web durante 30 segundos, por lo que podemos ver que está correctamente implementado.

HAProxy

```
backend backend_algarza04
  option httpchk GET /
  default-server inter 5s fall 3 rise 2
  server web1 192.168.10.2:80 maxconn 32 weight 1 check
  server web2 192.168.10.3:80 maxconn 32 weight 1 check
  server web3 192.168.10.4:80 maxconn 32 weight 4 check
  server web4 192.168.10.5:80 maxconn 32 weight 2 check
  server web5 192.168.10.6:80 maxconn 32 weight 1 check
  server web6 192.168.10.7:80 maxconn 32 weight 8 check
  server web7 192.168.10.8:80 maxconn 32 weight 1 check
  server web8 192.168.10.9:80 maxconn 32 weight 1 check
```

Le he añadido los parámetros:

- option httpchk GET / : esta primera opción se encarga de habilitar las comprobaciones de salud, el balanceador enviará una solicitud a cada servidor que comprobará si está funcionando correctamente.
- default-server inter 5s fall 3 rise 2: esta línea establece los parámetros para las comprobaciones de salud, el funcionamiento será el siguiente, se enviarán comprobaciones de salud cada 5s, si un servidor falla 3 veces consecutivas pasará a estar en estado inactivo, volverá a estar activo si completa 2 comprobaciones correctas de manera consecutiva
- El check de cada servidor es activar las comprobaciones para cada uno de los servidores.

Vamos a pasar a comprobar el funcionamiento de esta medida, en primer lugar he hecho el docker compose y he mandado una solicitud al balanceador, este le ha pasado la solicitud a web6 debido a que era el que mayor peso tenía, a continuación voy a desactivar este servidor con docker stop web6 y voy a ver que pasa.

web6	0	0	-	0	1	0	1	32	1	1	1m40s	616	355	0	0	0	0	0	2m3s UP	L7OK/200 in 1ms	8/8	Y
web6	0	0	-	0	1	0	1	32	1	1	3m48s	616	355	0	0	0	0	0	4m11s UP 2/3	L4TOUT in 5002ms	8/8	Y
web6	0	0	-	0	1	0	1	32	1	1	4m28s	616	355	0	0	0	0	0	25s DOWN	L4CON in 3061ms	8/8	Y
web6	0	0	-	0	1	0	1	32	1	1	5m13s	616	355	0	0	0	0	0	1m10s DOWN 1/2	L7OK/200 in 1ms	8/8	Y
web6	0	0	-	0	1	0	1	32	1	1	5m40s	616	355	0	0	0	0	0	22s UP	L7OK/200 in 1ms	8/8	Y

Como podemos ver en las capturas anteriores estos son los estados por los que ha pasado el servidor web6, en primer lugar al apagarlo empezó a mandar solicitudes, como este no las recibía cambió de color a amarillo y nos informó de que no proceso las dos últimas solicitudes que se le pasaron, después de esto perdió otra solicitud más y paso a estar caído. Una vez comprobado esto volví a levantar el servidor y como podemos ver nos indica que ha procesado una solicitud, como después de esto lo ha hecho con otra más ha pasado a estar UP nuevamente.

Tareas avanzadas - A5. Automatización de escalado del escenario

```
alvarogz@gagz-ubu24 ~/Escritorio/SWAP/practicas/P2/P2-algarza04-nginx-automatizado$ docker compose up -d
[+] Running 13/13
 ✓ Network p2-algarza04-nginx-automatizado_red_monitoreo Created
 ✓ Container prometheus Started
 ✓ Container web7 Started
 ✓ Container web8 Started
 ✓ Container web1 Started
 ✓ Container cadvisor Started
 ✓ Container web2 Started
 ✓ Container web5 Started
 ✓ Container web4 Started
 ✓ Container grafana Started
 ✓ Container web3 Started
 ✓ Container web6 Started
 ✓ Container nginx Started
```

CONTAINER ID	IMAGE	COMMAND	CREATED	STATUS	PORTS
f72a151a666a	algarza04-nginx-image:p2	"/docker-entrypoint..."	6 seconds ago	Up 4 seconds	0.0.0.0:8090->80/tcp, [::]:8090->80/tcp
nginx					
17489348d076	gcr.io/cadvisor/cadvisor:latest	"/usr/bin/cadvisor -..."	6 seconds ago	Up 5 seconds (health: starting)	0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
cadvisor					
1d0b3810381b	grafana/grafana:latest	"/run.sh"	6 seconds ago	Up 5 seconds	0.0.0.0:3000->3000/tcp, [::]:3000->3000/tcp
grafana					
7a6a9817ae4a	prom/prometheus:latest	"/bin/prometheus --c..."	6 seconds ago	Up 5 seconds	0.0.0.0:9090->9090/tcp, [::]:9090->9090/tcp
prometheus					
895f9fc60915	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8084->80/tcp, [::]:8084->80/tcp
web4					
171fcd9c66e1	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8082->80/tcp, [::]:8082->80/tcp
web2					
eba595ec0144	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8086->80/tcp, [::]:8086->80/tcp
web6					
19b138575029	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8088->80/tcp, [::]:8088->80/tcp
web8					
3e281a2b960	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8081->80/tcp, [::]:8081->80/tcp
web1					
8418805e8593	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8087->80/tcp, [::]:8087->80/tcp
web7					
8fbde4c8a5d2	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8085->80/tcp, [::]:8085->80/tcp
web5					
e7647bf47f21	algarza04-apache-image:p2	"/bin/sh -c 'apache2..."	6 seconds ago	Up 5 seconds	0.0.0.0:8083->80/tcp, [::]:8083->80/tcp
web3					

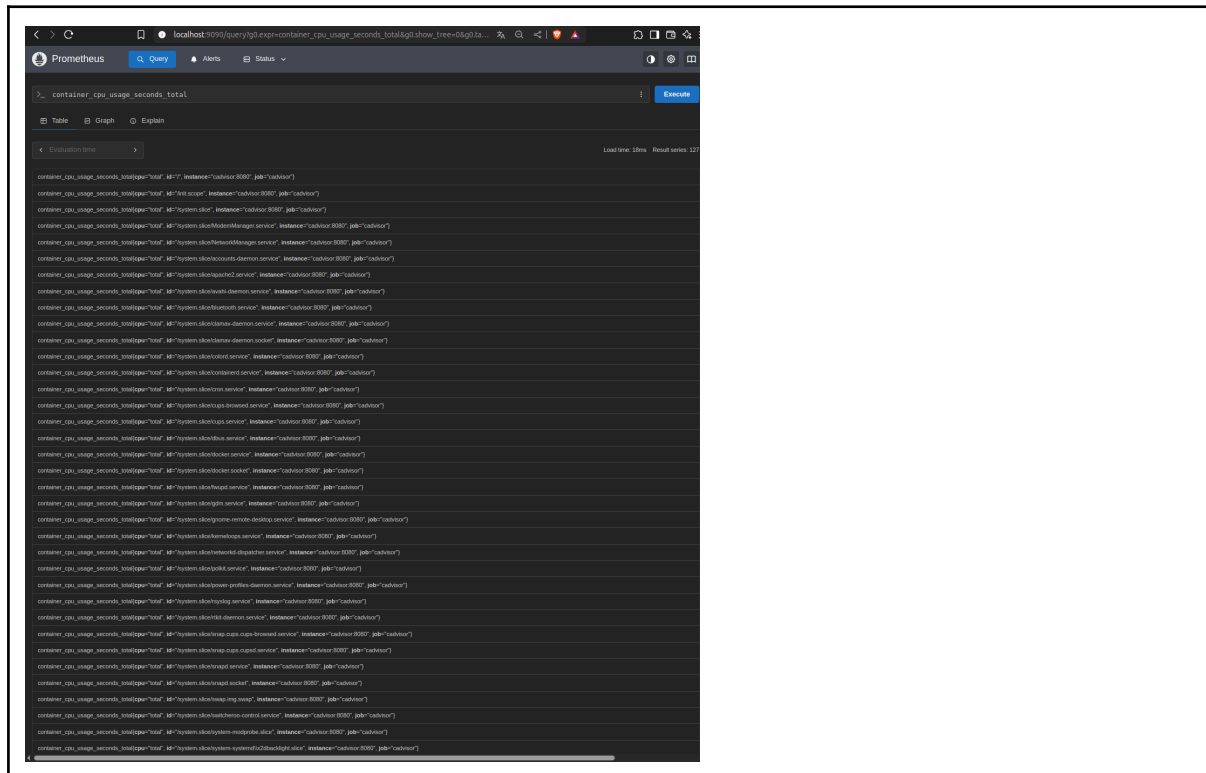
Aunque al principio me fallaba porque me faltaba agregar cosas he conseguido levantar todos los servidores.

Voy a pasar a comprobar si se ejecutan los contenedores.

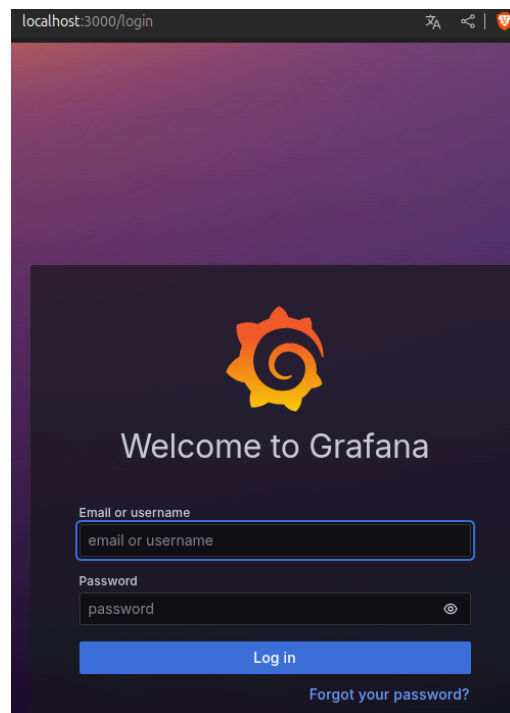


The screenshot shows the cAdvisor web interface in a browser window. At the top, there's a navigation bar with links for 'root', 'Docker Containers', 'Podman Containers', and 'Subcontainers'. Below this, there are sections for 'Isolation' (CPU, Memory, Swap Limit), 'Usage' (Overview, Processes, CPU, Memory), and 'Usage Breakdown'. The 'Overview' section features several circular gauges for CPU, Memory, and Swap usage. The 'CPU' section shows a line graph of 'Total Usage' over time. The 'Memory' section shows a line graph of 'Total Usage' over time. The 'Usage Breakdown' section shows a line graph of 'Usage Breakdown' over time.

Como podemos ver cadvisor funciona correctamente.



Prometheus también parece estar funcionando correctamente.



Para acceder el user y la contraseña es admin admin

Aquí podemos ver que he conseguido poner a funcionar todos los contenedores, aún así no he podido terminar el ejercicio debido a que cuando intenté que Grafana mostrara las métricas de Prometheus no me dejaba importarle el Dashboard, problema que hizo que no pudiera continuar realizando el ejercicio.

ANÁLISIS PROPUESTA IA

enlace al chat: <https://chatgpt.com/c/680615f0-6a2c-800d-86ac-ba956440ce22>

A continuación voy a pasar a hablar un poco de como me he ayudado de la ia para realizar esta práctica, en esta parte voy a dar una visión de la manera en que la he utilizado, ya que hay muchos ejercicios en los que ya he explicado zonas en las que he requerido de la ayuda de la IA.

A la hora de realizar los ejercicios básicos no he requerido mucho de su ayuda, simplemente para conocer algún parámetro de algún dockerfile o algo del .conf y para conocer para que servían ciertos aspectos del .conf. Por otro lado al pasar a las tareas avanzadas me ha sido muy útil, para esta parte la he utilizado para ayudarme a completar algunos archivos de configuración o para saber como añadirles cosas a estos, como podría ser aplicarle el balanceo por pesos o por menor tiempo. Al igual que con las básicas también la he utilizado para comprender algunos conceptos, como por ejemplo el funcionamiento de Traefik.

A pesar de que me ha ayudado mucho, la ia es una herramienta que hay que saber manejar para que te pueda ayudar, hay veces que no te es capaz de dar buenas explicaciones a lo que le estas preguntando o te dice que algo está correcto cuando no es así, como cuando me decía que no era necesario aplicar todas las redes a Traefik, cosa que cuando no lo hacías dejaba de funcionar el balanceador. Al usarla hay que tener autocrítica y coger solo lo que sea necesario y beneficioso para tu caso en particular, un último ejemplo de esto es que al hacer los archivos de Traefik no le crea un Dockerfile, cosa que es muy recomendable por si en un futuro hay que aumentar su funcionamiento.

Con todo esto podemos sacar como conclusión que si conseguimos aprovecharla al máximo será la mejor herramienta que podemos utilizar, independientemente de que no haya que hacer todo lo que diga y como lo diga.